

# CXU Playground

## RISC-V Custom Extension Development Framework

Petro Mozil

Ukrainian Catholic University

2026

# Motivation – why use extension plugins at all?

## RISC-V extensions

- ▶ RISC-V is designed with extensions in mind
- ▶ The instruction set allows for many custom commands
- ▶ Official extensions are managed by the RISC-V committee [riscv-spec]
- ▶ These extensions are hardly trivial to design – a lot must be kept in mind (memory ordering, compute constraints etc.)
- ▶ Extensions may share opcodes – this breaks compatibility between some extensions
- ▶ Extensions are hard to publish – it takes a while to ratify the extension

# Motivation – why CXU?

## CFU - exists

- ▶ Reusable across RISC-V cores
- ▶ Up to 255 functions
- ▶ CFU Playground: ready sandbox

One extension per thread,  
assembly-only

## CXU — spec only

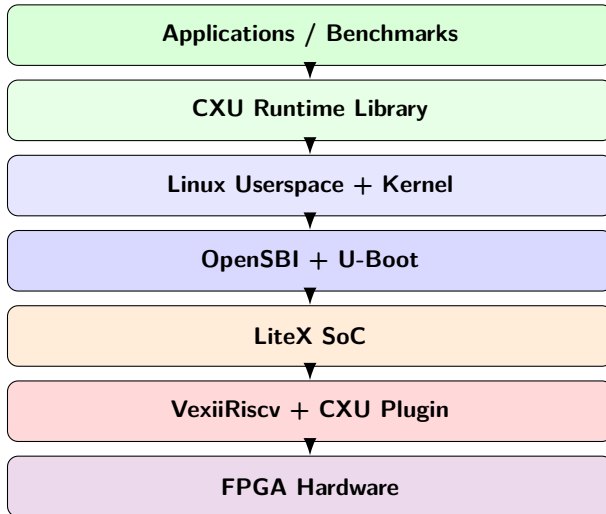
- ▶ Multiple simultaneous extensions
- ▶ Shared state
- ▶ C-level ABI, no inline assembly

Spec only for the extension, no  
integration yet

## This work

- ▶ First CXU hardware
- ▶ Full-stack open-source sandbox
- ▶ Benchmarked under Linux

# Architecture



# SpinalHDL

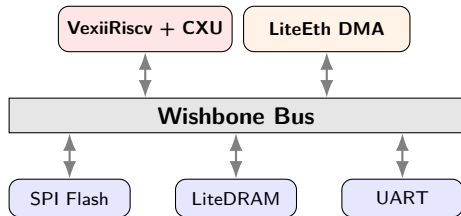
## What it is

- ▶ Hardware description language built on Scala [`@spinalhdl`]
- ▶ Generates Verilog/VHDL
- ▶ Modular plugin system: no plugin may alter the behaviour of another one

## CXU plugin enable

```
VexiiRiscvLitex(  
    plugins += CxuPlugin(...args)  
)
```

## LiteX SoC Framework (enjoy-digital 2025)



# Target hardware

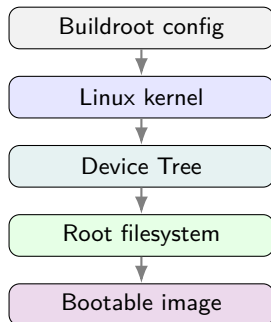
## Target: Arty S7-50

- ▶ Spartan-7 XC7S50
- ▶ 50 MHz, 256 MB DDR3

## FPGA utilisation (With AES example)

- ▶ CPU + SoC: 23%
- ▶ Single CXU extension:  $\approx 1\%$
- ▶ Rest: free

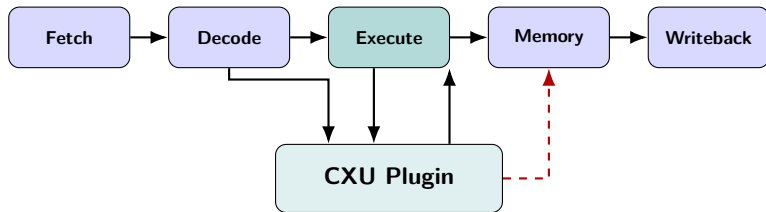
# Buildroot (Mozil 2026a)



## Boot sequence

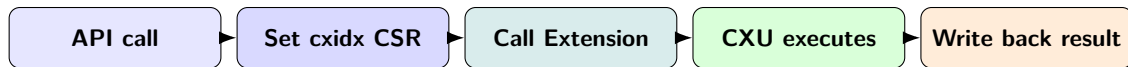
LiteX BIOS → OpenSBI → U-Boot → Linux → CXU app

## CXU Plugin



`cxidx` CSR selects active extension. Stalls pipeline for multi-cycle ops.

## CXU Instruction Execution Flow



# CXU Runtime Library

## Without the library

```
// Manual extension select
csrw cxidx, 2

// Custom opcode call (most compilers don't
support all RISC-V extensions)
.insn r CUSTOM_0, 0, 0,
    a0, a1, a2

// Core-specific, not portable
```

## With the runtime library

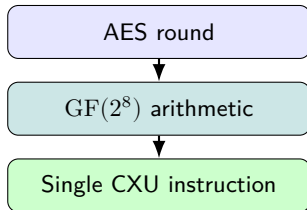
```
// Pure C --- no assembly
cxu_select(CXU_AES);
uint32_t r = cxu_call(a, b);

// Portable across CXU cores
```

# Benchmarks

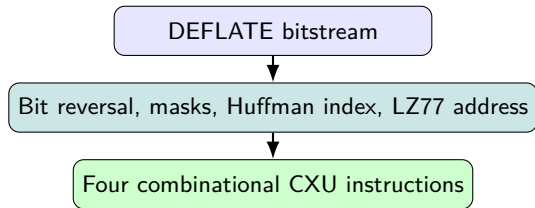
- ▶ **Bare-metal:** `mcycle` / `minstret` counters
- ▶ **Linux:** timing with utils provided by POSIX time utils
- ▶ Multiple input sizes to separate fixed overhead from per-byte savings
- ▶ Compiler flags identical (force only `-O3`) for both baseline and CXU versions

## Case Study: AES GF( $2^8$ ) (Marshall et al. 2020)



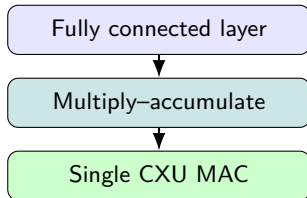
**5.25**  $\times$  speedup

## Case Study: GZIP / DEFLATE (Mozi 2026b)



**10% end-to-end speedup**

## Case Study: TFLite Micro MAC (David et al. 2021)



**13% compute speedup**

## FPGA Resource Usage (Arty S7-50, XC7S50)

| Extension | Logic cells | CXU cost | DSP48E1 | Block RAM |
|-----------|-------------|----------|---------|-----------|
| AES       | 20,948      | 209 (1%) | 4       | 36 tiles  |
| GZIP      | 20,666      | 20       | 4       | 36 tiles  |
| TFLite    | 20,698      | folded   | 4       | 36 tiles  |
| All three | 21,394      | 229      | 4       | 36 tiles  |

*A single CXU extension will likely (unless it's very complex) consume little fpga fabric (~1% or less)*

## How to Add a New Extension

- ▶ Create the extension with the interface provided in examples (do mind - the names of the module go as 'Cxu0', 'Cxu1', and so on)
- ▶ Compile the system-on-chip with the created extension (add a cli argument: '-cxu <path-to-cxu-source>' to the build command)
- ▶ **Note - there's no Device Tree entries:** extension addresses hard-coded.

## Limitations

- ▶ **CSR access latency:** `cxidx/cxdata` are Wishbone-mapped, not pipeline registers.
- ▶ **No context-switch support:** CXU state not saved on Linux task preemption — single-process only.
- ▶ **No Device Tree entries:** extension addresses hard-coded.
- ▶ **Clock frequency:** combinational paths reduce  $F_{\max}$ .

## Future Work

- ▶ Process context-switch support (save/restore CXU state)
- ▶ Native pipeline CSRs (bypass Wishbone)
- ▶ Device Tree integration (dynamic discovery)
- ▶ Multi-process tests and benchmarks on Linux

# Summary

- ▶ CXU hardware implementation (SpinalHDL plugin for VexiiRiscv)
- ▶ CXU Playground: open-source, full-stack (LiteX + Buildroot + runtime lib)
- ▶ Three extensions that offer significant speedup: AES ( $5.25\times$ ), GZIP ( 10%), TFLite MAC ( 13%)

<https://github.com/pmozil/CXU-playground>

<https://github.com/pmozil/vexiiriscv>

<https://github.com/pmozil/cxu-buildroot>

## Similar extension interfaces / playgrounds

| Feature      | CFU Playground(Prakash et al. 2022) | FRANCIS-V(Egert 2023)   | CXU Playground (this work)                                        |
|--------------|-------------------------------------|-------------------------|-------------------------------------------------------------------|
| Interface    | CFU (single extension)              | Core-V-XIF              | CXU (multiple extensions)                                         |
| Invocation   | Assembly (custom-0/1/2/3)           | C with compiler support | Pure C, portable API                                              |
| Shared state | No                                  | Yes (custom CSRs)       | Yes (cxdata CSR)                                                  |
| OS support   | Bare-metal only                     | Bare-metal + RTOS       | <b>Full Linux (Buildroot)</b>                                     |
| Portability  | Across cores                        | CPU-specific            | One core for now, designed for cross-core reuse as CFU playground |

# References

- David, Robert, Jared Duke, Advait Jain, Vijay Janapa Reddi, Nat Jeffries, Jian Li, Nick Kreeger, et al. 2021. "TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems." In *Proceedings of Machine Learning and Systems*, 3:800–811. <https://arxiv.org/pdf/2010.08678>.
- Egert, Florian. 2023. "FRANCIS-v: Framework for Integrating Custom Instructions into RISC-v Systems." Master's thesis, TU Wien. <https://repositum.tuwien.at/bitstream/20.500.12708/188868/1/Egert%20Florian%20-%202023%20-%20FRANCIS-V%20FRAMework%20for%20iNtegrating%20Custom%20InstructionS...pdf>.
- enjoy-digital. 2025. "LiteX: Build Your Hardware, Easily!" <https://github.com/enjoy-digital/litex>.
- Marshall, Ben, G. Richard Newell, Dan Page, Markku-Juhani O. Saarinen, and Claire Wolf. 2020. "The Design of Scalar AES Instruction Set Extensions for RISC-V." *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2021 (1): 109–36. <https://doi.org/10.46586/tches.v2021.i1.109-136>.
- Mozil, Petro. 2026a. "Cxu-Buildroot: Buildroot Support for CXU-Enabled Systems." <https://github.com/pmozil/cxu-buildroot>; GitHub.
- . 2026b. "Gzip: Sample GZIP Compression Implementation for Accelerator Studies." <https://github.com/pmozil/gzip>; GitHub.
- Prakash, Shvetank, Tim Callahan, Joseph Bushagour, Colby Banbury, Alan V. Green, Pete Warden, Tim Ansell, and Vijay Janapa Reddi. 2022. "CFU Playground: Full-Stack Open-Source Framework for Tiny Machine Learning (tinyML) Acceleration on FPGAs." *arXiv Preprint*. <https://arxiv.org/abs/2201.01863>.